

```

        host1="192.168.142.130"; //VM2 as per Figure 2
        host2 = "192.168.142.131"; //VM1 as per Figure 2
        minmax_1 (host1,host2,x,10);
        exit (0);
}

/* This is dist_app_sever1.c generated by rpcgen and modified
according to the need */

#include "dist_app.h"
int * max_1_sec(ipair *argp, struct sec_req *rqstp) // The
service Max definition
{
    static int result;
    int i,big;
    printf("\n Got request for finding the max
element in the given array\n");
    big=argp->data[0];
    for(i=1;i<argp->count;i++)
        if(big<argp->data[i])
            big=argp->data[i];
    result=big;
    return &result;
}
int * min_1_sec(ipair *argp, struct sec_req *rqstp) // This
is not used here as it is not required.
{
    static int result;
    /*
    * insert server code here
    */
    return &result;
}
/* This is dist_app_sever2.c generated by rpcgen and
modified according to the need */
#include "dist_app.h"

int * max_1_sec(ipair *argp, struct sec_req *rqstp) // This
is not used here as it is not required.
{
    static int result;
    /*
    * insert server code here
    */
    return &result;
}
int * min_1_sec(ipair *argp, struct sec_req *rqstp) // The
service Min Definition
{
    static int result;

    int i,small;
    printf("\n Got request for finding the min element in the
given array\n");
    small=argp->data[0];

```

```

root@client-virtual-machine: /home/client/Desktop/RPC
root@client-virtual-machine:/home/client/Desktop/RPC# ./client
give any 10 elements 10 11 12 13 14 211 54 88 99 15
The result from the Service Max is :211
The result from the Service Min is :10
root@client-virtual-machine:/home/client/Desktop/RPC# █

```

Figure 6: Executing the client program in VM3

```

root@kvsrv-virtual-machine: /home/kvsrv/RPC
root@kvsrv-virtual-machine:/home/kvsrv/RPC# ./service_max
Got request for finding the max element in the given array

```

Figure 7: Executing the service *max* in VM2

```

root@gratan-virtual-machine: /home/ratan
root@gratan-virtual-machine:/home/ratan# ./service_min
Got request for finding the min element in the given array

```

Figure 8: Executing the service *min* in VM1

```

for(i=1;i<argp->count;i++)
    if(small>argp->data[i])
        small=argp->data[i];

    result=small;
    return &result;
}

```

Executing the distributed application

We can execute the distributed application as shown in Figures 6, 7 and 8, respectively. The above modified code can also be downloaded from http://opensourceforu.com/article_source_code/jan2016/rpcgen.zip

Developing a distributed application is simple and easy with RPC, as compared to socket programming. Developers can use *rpcgen* to generate sample client and server programs, and then modify them easily, according to requirements. 

References

- [1] Distributed Operating Systems (1st Edition) by Andrew S. Tanenbaum
- [2] https://en.wikipedia.org/wiki/Open_Network_Computing_Remote_Procedure_Call
- [3] <https://en.wikipedia.org/wiki/RPCGEN>
- [4] <https://docs.freebsd.org/44doc/psd/22.rpcgen/paper.pdf>
- [5] Power Programming with RPC by John Bloomer, Publisher: O'Reilly Media
- [6] Implementing Remote Procedure Calls by Andrew D. Birrell and Bruce Jay Nelson, Xerox, Palo Alto Research Center

By: S. Ratan Kumar

The author currently works as the incubation centre coordinator at Anil Neerukonda Institute of Technology & Sciences (ANITS), Andhra Pradesh. He can be reached at sratankumar.cse@anits.edu.in. Web page: <https://sites.google.com/site/sajjaratankumar/>